FAST National University of Computer and Emerging Sciences

Karachi Campus

Department of Computer Science

FINAL PROJECT REPORT

Theme Park Management System

An Object-Oriented C++ Application Implementing Inheritance, Polymorphism, File Handling, Templates, and Exception Handling

Course:	Object Oriented Programming (OOP)
Instructor:	Sir Muhammad Khalid Khan
Section:	BAI-2C
Submission Date:	April 2026
Group Members	
Member 1:	Mohammad Sarim Raza **25K-0075**
Member 2:	Abbas Ali Haider **25K-0009**

Table of Contents

1. Executive Summary **3**

2. Introduction **3**

2.1 Background & Motivation 3

2.2 Problem Statement 3

2.3 Objectives 4

2.4 Scope 4

3. System Analysis **4**

3.1 Functional Requirements 4

3.2 Non-Functional Requirements 5

4. Object-Oriented Design **5**

4.1 Class Hierarchy & UML Diagram 5

4.2 Class Descriptions 6

4.3 Relationships Between Classes 7

5. Implementation **7**

5.1 OOP Concepts Implemented 7

5.2 Code Snippets 8

5.3 File Structure 11

6. Testing & Results **11**

7. Challenges & Solutions **12**

8. Division of Work **12**

9. Conclusion & Future Work **13**

10. References **13**

1. Executive Summary

This report presents the complete design and implementation of the **Theme Park Management System**, a C++ console application developed for the Object Oriented Programming (OOP) Lab at FAST National University of Computer and Emerging Sciences, Karachi, under the supervision of Sir Muhammad Khalid Khan.

The system models a real-world theme park using a rich class hierarchy. It demonstrates all fundamental and advanced OOP concepts including encapsulation, inheritance, polymorphism, abstract classes, operator overloading, templates, exception handling, and file I/O. The project was developed by two members, Mohammad Sarim Raza (25K-0075) and Abbas Ali Haider (25K-0009). The final application allows staff to register visitors and book tickets, while the manager can add rides, events, staff, inventory, view feedback, and generate revenue reports — all persisted to files across sessions.

2. Introduction

2.1 Background & Motivation

Theme parks involve many daily operations such as visitor registration, ride management, event scheduling, ticket booking, staff management, feedback collection, inventory control, and revenue tracking. Handling these operations manually can lead to data inconsistency, poor record keeping, and difficulty in managing the park efficiently. This project was created to model these operations using Object Oriented Programming concepts in C++.

This project also serves as a practical exercise in applying all OOP concepts studied during the semester in a single application.

2.2 Problem Statement

The core problems addressed by this project are:

- No automated way to track visitors, rides, events, and tickets.
- Manual management of staff, inventory, and feedback records.
- Revenue calculation is difficult without a structured software system.
- Data is lost between sessions without persistent storage.
- Operations need role-based access so staff and manager can do different tasks.

2.3 Objectives

1. Design a well-structured class hierarchy using inheritance and polymorphism.

2. Implement all core OOP concepts: encapsulation, inheritance, polymorphism, abstraction, operator overloading, templates, and exception handling.
3. Persist all data using C++ file handling (fstream).
4. Provide a clean, menu-driven interface for staff and manager operations.
5. Implement login-based access for two roles with one staff user and one manager user.

2.4 Scope

- **In Scope:** Visitor management, ride management, event management, ticket booking, staff records, feedback, inventory, revenue report, file persistence.
- **Out of Scope:** GUI interface, database connectivity, online booking, mobile app integration.

3. System Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Staff can register new visitors with unique ID.	High
FR-02	Staff can view rides and events available in the park.	High
FR-03	Staff can book tickets for visitors.	High
FR-04	Manager can add staff, rides, events, and inventory items.	High
FR-05	Manager can view feedback and revenue reports.	High
FR-06	System saves and loads all data from files automatically.	High
FR-07	System supports role-based login for staff and manager.	High
FR-08	System displays all records in formatted console output.	Medium
FR-09	System calculates total revenue based on ticket sales.	Medium
FR-10	System handles invalid input and missing records using exceptions.	High

3.2 Non-Functional Requirements

- **Code Quality:** Classes must be clear and easy to understand.
- **Robustness:** Invalid inputs must be handled safely.
- **Maintainability:** The code should be structured in a readable manner.
- **Portability:** Must compile with g++ on Windows.
- **Persistence:** Data should remain available after the program closes.

4. Object-Oriented Design

4.1 Class Hierarchy & UML Diagram

The application is built around the following class hierarchy:

```
<<abstract>> ParkEntity / | | | \ / | | | \ Visitor Ride Event Staff
Inventory Ticket -- operator overloading / friend functions Feedback --
aggregation / average rating User -- authentication and roles
ThemeParkSystem -- main controller class
```

4.2 Class Descriptions

ParkEntity (Abstract Base Class)

Abstract base class representing any park entity. Contains protected attributes: ID and name. Declares pure virtual methods `display()` and `serialize()` so every derived class provides its own implementation.

Visitor (Derived from ParkEntity)

Represents a park visitor. Adds age and email attributes and uses a static ID counter for automatic ID generation.

Ride (Derived from ParkEntity)

Represents a ride in the park. Stores price and operational status, and allows the system to show whether the ride is available or under maintenance.

Event (Derived from ParkEntity)

Represents a park event such as Eid Festival, Bake Sale, Chand Raat, Basant Kite Festival, and Holi Color Festival. Stores date, capacity, and number of registered visitors.

Staff (Derived from ParkEntity)

Represents staff members working in the park. Stores role and salary information.

Inventory (Derived from ParkEntity)

Represents park inventory items such as popcorn, ice cream, soft drinks, cotton candy, and merchandise.

Ticket

Stores ticket booking information and supports discount logic and output formatting.

Feedback

Stores visitor feedback and rating, and supports average rating calculation through a friend function.

User

Handles login credentials and role-based authentication for staff and manager users.

ThemeParkSystem

The main controller class that handles menus, login, file loading, file saving, and all operations of the application.

4.3 Relationships Between Classes

Relationship	Classes Involved	Type
ParkEntity is the base of other park entities	ParkEntity, Visitor, Ride, Event, Staff, Inventory	Inheritance
ThemeParkSystem manages collections of objects	ThemeParkSystem, Visitor, Ride, Event, Staff, Inventory, Ticket, Feedback, User	Aggregation
Ticket uses friend function and operator overloading	Ticket, applyDiscount, operator<<	Association
Feedback uses friend function for average rating	Feedback, calculateAverageRating	Association
User controls login access	User, ThemeParkSystem	Dependency

5. Implementation

5.1 OOP Concepts Implemented

OOP Concept	Implementation Detail
Encapsulation	All important data members are private and accessed through public methods.
Abstraction	ParkEntity provides an abstract interface for all park records.
Inheritance	Visitor, Ride, Event, Staff, and Inventory inherit from ParkEntity.
Polymorphism	Virtual display() and serialize() functions are overridden in derived classes.
Abstract Classes	ParkEntity cannot be instantiated directly because it has pure virtual functions.
Operator Overloading	Ticket class overloads +, <, and << operators.
Templates	readNumber() and findById() are template functions.
Exception Handling	runtime_error and invalid_argument are used for invalid operations.
File Handling	All data is saved and loaded from text files.
STL Usage	vector, algorithm, numeric, and stringstream are used.
Friend Functions	applyDiscount() and calculateAverageRating() access private data when needed.
Static Members	Static nextId values generate unique IDs for each class.

5.2 Code Snippets

Abstract Base Class: ParkEntity

```
class ParkEntity
                {
                protected:
                int id;
                string name;

                public:
                ParkEntity(int id, const string &name) : id(id), name(name) {}
                virtual ~ParkEntity() {}

                int getId() const { return id; }
                string getName() const { return name; }

                virtual void display() const = 0;
                virtual string serialize() const = 0;
                };
```

Inheritance & Polymorphism: Visitor and Ride

```
class Visitor : public ParkEntity
                {
                private:
                int age;
                string email;
                static int nextId;

                public:
                Visitor(const string &name, int age, const string &email)
                : ParkEntity(nextId++, name), age(age), email(email) {}

                void display() const override
                {
                cout << "Visitor ID: " << id << " | Name: " << name << " | Age:
" << age << " | Email: " << email
                    << "\n" ; } }; class Ride : public ParkEntity { private:
double price; bool operational; public:
                    void display() const override { cout << "Ride ID: " << id <<
" | Name: " << name << " | Price: $" <<
                    fixed << setprecision(2) << price << " | Status: " <<
(operational ? "Operational" : "Maintenance" )
                    << "\n" ; } };
```

Operator Overloading: Ticket

```
class Ticket
                        {
```

```
                        private:
                        int id;
                        int visitorId;
                        int rideId;
                        double amount;

                        public:
                        double operator+(const Ticket &other) const { return
amount + other.amount; }
                        bool operator<(const Ticket &other) const { return
amount < other.amount; } friend void
                            applyDiscount(Ticket &ticket, double percentage);
friend ostream &operator<<(ostream &os,
                            const Ticket &ticket); };
```

File Handling

```
void saveUsers() const
                                {
                                ofstream uFile(usersFile);
                                for (const auto &u : users)
                                uFile << u.serialize() << "\n";
                                }

                                void loadUsers()
                                {
                                ifstream file(usersFile);
                                string line;
                                while (getline(file, line))
                                {
                                stringstream ss(line);
                                string userIdStr, username, password, role;
                                getline(ss, userIdStr, '|');
                                getline(ss, username, '|');
                                getline(ss, password, '|');
                                getline(ss, role, '|');
                                users.emplace_back(stoi(userIdStr), username,
password, role);
                                }
                                }
```

5.3 File Structure

File	Purpose
ThemeParkProject.cpp	Main source file with full system implementation
Visitors.txt	Visitor records
Rides.txt	Ride records

Tickets.txt	Ticket records
Events.txt	Event records
Staff.txt	Staff records
Feedback.txt	Feedback records
Inventory.txt	Inventory records
Users.txt	Login credentials

6. Testing & Results

6.1 Functional Test Cases

Test Case	Input / Action	Expected Result	Status
Login as Staff	staff1 / pass123	Staff portal opens	Pass
Login as Manager	manager1 / admin123	Manager portal opens	Pass
Register Visitor	Valid name, age, email	Visitor added and saved	Pass
Book Ticket	Valid visitor ID and ride ID	Ticket booked successfully	Pass
Add Event	Event name, date, capacity	Event added and saved	Pass
Add Staff	Name, role, salary	Staff added and saved	Pass
View Feedback	Open manager feedback menu	All feedback displayed with average rating	Pass
Revenue Report	Open revenue report menu	Total ticket revenue displayed	Pass
File Persistence	Close and reopen program	Data reloaded from files	Pass

6.2 Sample Console Output

```
===== THEME PARK MANAGEMENT SYSTEM =====
1. Login as Staff
2. Login as Manager
0. Exit

Enter choice:
```

```
--- STAFF PORTAL ---

=== Staff Menu ===
1. Register Visitor
2. View Available Rides
3. View Available Events
4. Book Ticket for Visitor
5. View All Visitors
6. Logout

Enter choice:
```

```
--- MANAGER PORTAL ---

=== Manager Menu ===
1. Add Staff Member
2. Add Ride
3. Add Event
4. Add Inventory Item
5. View Feedback
6. View Revenue Report
7. View All Staff
8. View All Rides
9. View All Events
10. View All Inventory
11. Save Data
12. Logout

Enter choice:
```

```
===== REVENUE REPORT =====
Total Ticket Revenue: $495.00
```

7. Challenges & Solutions

Challenge	Solution Applied

Managing multiple record types in one program	Used separate classes and separate vectors for each entity type.
Keeping file data persistent	Added saveData() and loadData() functions with text files.
Handling invalid user input	Implemented readNumber() template and exception handling.
Role-based access control	Created User class with role checking for Staff and Manager.
Keeping code organized	Separated logic into clear classes and methods.

8. Division of Work

Member	Roll No.	Modules / Tasks	Contribution
Mohammad Sarim Raza	25K-0075	Class hierarchy design, visitor/ride/event modules, polymorphism, report formatting	50%
Abbas Ali Haider	25K-0009	File handling, login system, ticket booking, feedback, inventory, testing	50%

> **Note:** Both members understand the complete codebase and can explain any part of the project during the viva evaluation.

9. Conclusion & Future Work

9.1 Conclusion

The Theme Park Management System successfully demonstrates all OOP concepts studied in the course within a single application. The class hierarchy clearly models the theme park domain with proper use of abstraction, encapsulation, inheritance, and polymorphism. The use of templates, custom exceptions, file handling, STL containers, operator overloading, and static members further reflects a complete application of C++ OOP features.

The project gave both team members practical experience with class design, pointer-based polymorphism, file handling, and multi-class system development.

9.2 Future Work

- Add delete and edit options for visitors, rides, events, and staff.
- Add search functionality for records.
- Add graphical user interface.
- Add more detailed reporting features.

- Add online ticket booking support.

10. References

1. C++ Programming Language Documentation
2. Object Oriented Programming course notes
3. STL and file handling references

Group Signature

Instructor Approval
Sir Muhammad Khalid Khan